ENTERPRISE ARCHITECTURAL SPECIFICATION • V2.0

DevToolBox SaaS Platform

Master Architectural Blueprint, Figma Screen Design Specifications, Entity-Relationship Models, and React 19 / Spring Boot 3.4 Implementation Roadmap.

TARGET BACKEND ARCHITECTURE
Spring Boot 3.4 • Spring Security 6 • Spring AI • Redis • Postgres

TARGET FRONTEND ARCHITECTURE
React 19 • Tailwind CSS v4 • Vite • Framer Motion • Recharts

PRIMARY PERSONAS
Normal End-Users • Software Developers • System Admins

AUTONOMOUS OBSERVABILITY
Spring AI Incident Diagnosis & Auto-Ticketing Agent

1. Executive Audit & SaaS Architecture Strategy

DevToolBox is evolving from a single-tier Spring Framework WAR into an industry-grade, **Dual-Track SaaS Ecosystem**. It provides both frictionless browser-based productivity utilities for normal users and high-throughput, low-latency REST APIs for external developers.

> **Core Modernization Goals**
>
> Migrate from stateful servlet sessions (`HttpSession`) to stateless JWT & API Key authentication, implement distributed Redis rate-limiting (token bucket), containerize native PDF/Word converters with Docker, integrate Spring AI for automated incident triage, and build a modular React 19 frontend with Tailwind CSS.

1.1 Modernization Matrix

SYSTEM LAYER	CURRENT LEGACY IMPLEMENTATION	TARGET INDUSTRY-GRADE SAAS
Packaging & Runtime	Spring MVC 6.1 WAR requiring external Tomcat servlet container.	Standalone **Spring Boot 3.4.x JAR** running on embedded Tomcat / Undertow.
Authentication	Stateful `HttpSession`; in-memory `ConcurrentHashMap` for OTPs.	Stateless **Spring Security 6 + JJWT** + Redis OTP store + OAuth2 (Google/GitHub).
Developer Access	None. Tools only callable through web forms.	**API Key Infrastructure (`X-API-KEY`)** with SHA-256 hashed secrets and usage analytics.
Rate Limiting	None. Platform vulnerable to scraping & DDoS.	**Redis Token Bucket Rate Limiter** per API key and per IP address.
Database & DDL	Static MySQL dump; Hibernate `hbm2ddl.auto=update`.	**PostgreSQL** with version-controlled **Flyway** database migrations.
Observability & AI	Hardcoded Gemini endpoint for basic chat.	**Spring AI Autonomous Agent**: Monitors latency/errors and generates Admin Incident Tickets.
PDF / File Native Tools	Hardcoded Windows paths (`C:\Program Files\gs\...`).	**Multi-stage Dockerfile** with Linux Ghostscript and headless LibreOffice.

1.2 User Flow Architecture & Decision Framework

Architectural Recommendation on Role Selection Placement:
"Should the screen asking whether the user is a Normal User or Developer be before or after sign-up, or after login?"

Industry Best Practice Decision: Pre-Filtered Intent + In-Modal Persona Selector

1. On the Landing Page: Two explicit call-to-actions funnel visitors immediately: `Launch Web Studio` sets intent to `NORMAL_USER`, while `Get API Keys & Docs` sets intent to `DEVELOPER`.

2. On Sign Up (The Decision Card): If a visitor clicks general "Sign Up", the modal presents an active toggle at the very top: `[ Normal User ]` vs `[ Developer ]`.

- Selecting **Normal User** enables: **Google OAuth** and **Email/Password**. (GitHub button is hidden to keep it simple for casual creators).
- Selecting **Developer** enables: **GitHub OAuth**, **Google OAuth**, and **Email/Password**. (Developers strongly prefer 1-click GitHub auth).

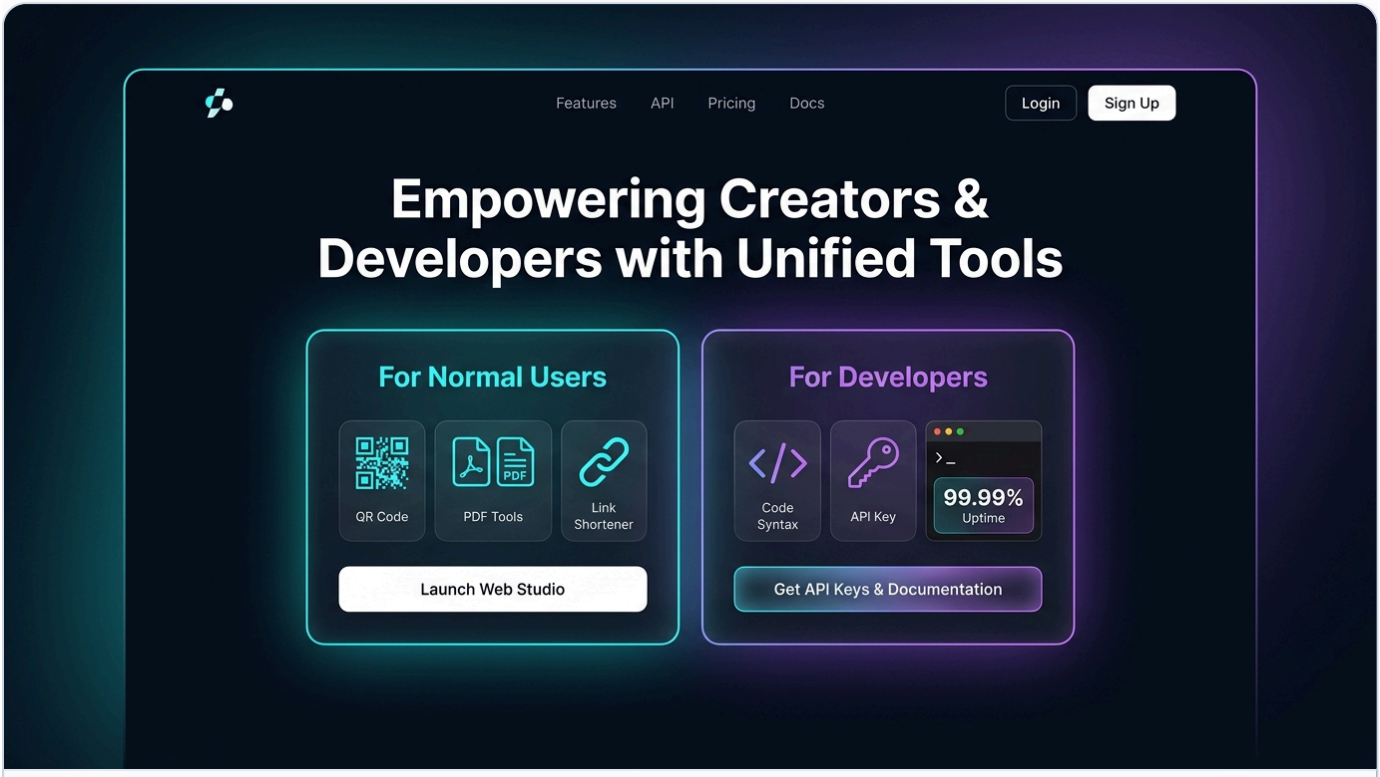
3. On Login (Zero-Friction): The user simply logs in. The backend JWT payload contains the stored role claim: `role: "ROLE_USER" | "ROLE_DEVELOPER" | "ROLE_ADMIN"`. The React router automatically redirects the user to their designated dashboard without asking them again!

2. Figma Screen Blueprints & UI/UX Specifications

The DevToolBox design system is built upon a **Cyber-Modern Dark Theme** utilizing Tailwind CSS v4, subtle neon borders (cyan `#06b6d4` and violet `#8b5cf6`), glassmorphic background cards (`backdrop-blur-md`), and clean, high-contrast typography.

Screen 1: Landing Page & Dual-Track Role Entry

Greets the visitor with the core platform identity and allows them to branch into their preferred experience immediately.



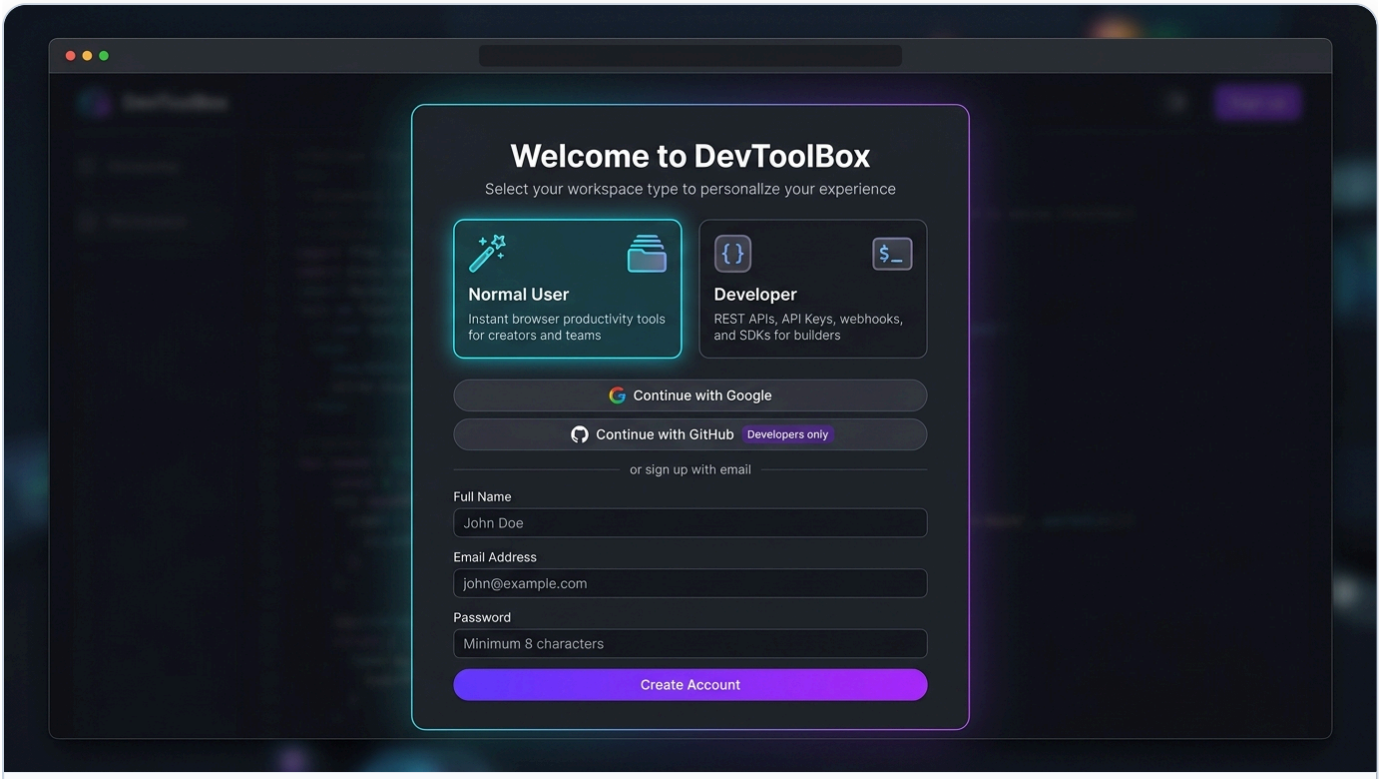
Screen 1: Landing Page with Interactive Role Cards ROUTE: /

Figma Component Structure:

- **Header Navigation:** Logo, Navigation Links (Features, API, Pricing, Docs), Theme Toggle, `Login` & `Sign Up` buttons.
- **Hero Title:** *"Empowering Creators & Developers with Unified Tools"* with gradient text clipping.
- **Left Card (Normal User):** Tool icons (QR, PDF, URL), feature list, and glowing CTA: `Launch Web Studio`.
- **Right Card (Developer):** Terminal badge (`99.99% Uptime`), code syntax brackets, API key icon, and CTA: `Get API Keys & Documentation`.

Screen 2: Authentication & Role Selection Modal

Unifies registration, workspace selection, and differentiated OAuth options (Google vs GitHub) in a single high-conversion interface.



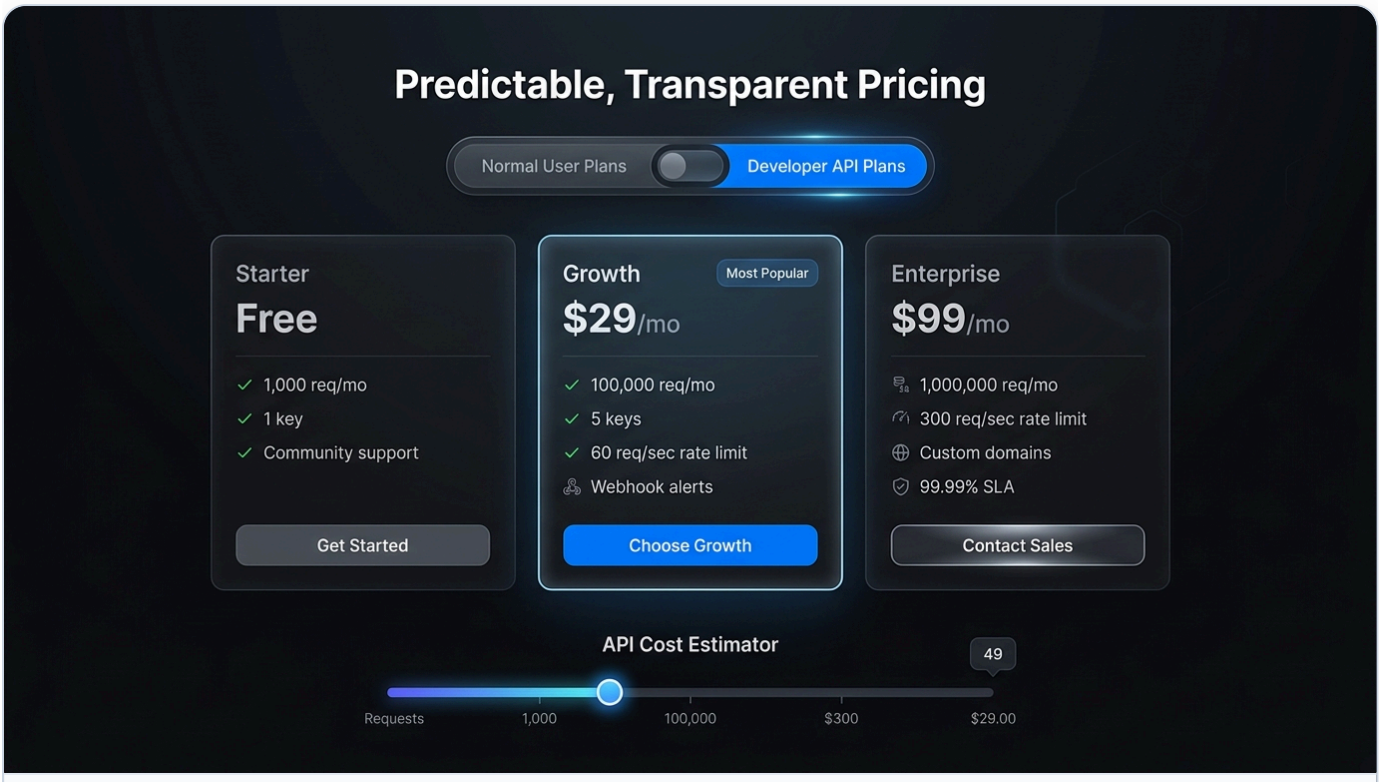
Screen 2: Unified Authentication & Workspace Persona Selector ROUTE: /SIGNUP & /LOGIN

Figma Component Structure:

- **Workspace Selector (Top):** Interactive radio cards for *"Normal User"* (browser tools) and *"Developer"* (APIs & SDKs).
- **OAuth Button Stack:**
  - `Continue with Google` (Available for both user types).
  - `Continue with GitHub` (Dynamically displayed with badge *"Developers only"* when Developer role is selected).
- **Email/Password Fields:** Full Name, Work Email, Password with real-time strength meter.
- **Submit CTA:** Purple gradient button: `Create Account` or `Sign In`.

Screen 3: Dual-Track Pricing Matrix

Provides clear pricing tiers tailored specifically for browser end-users versus API consumption developers.



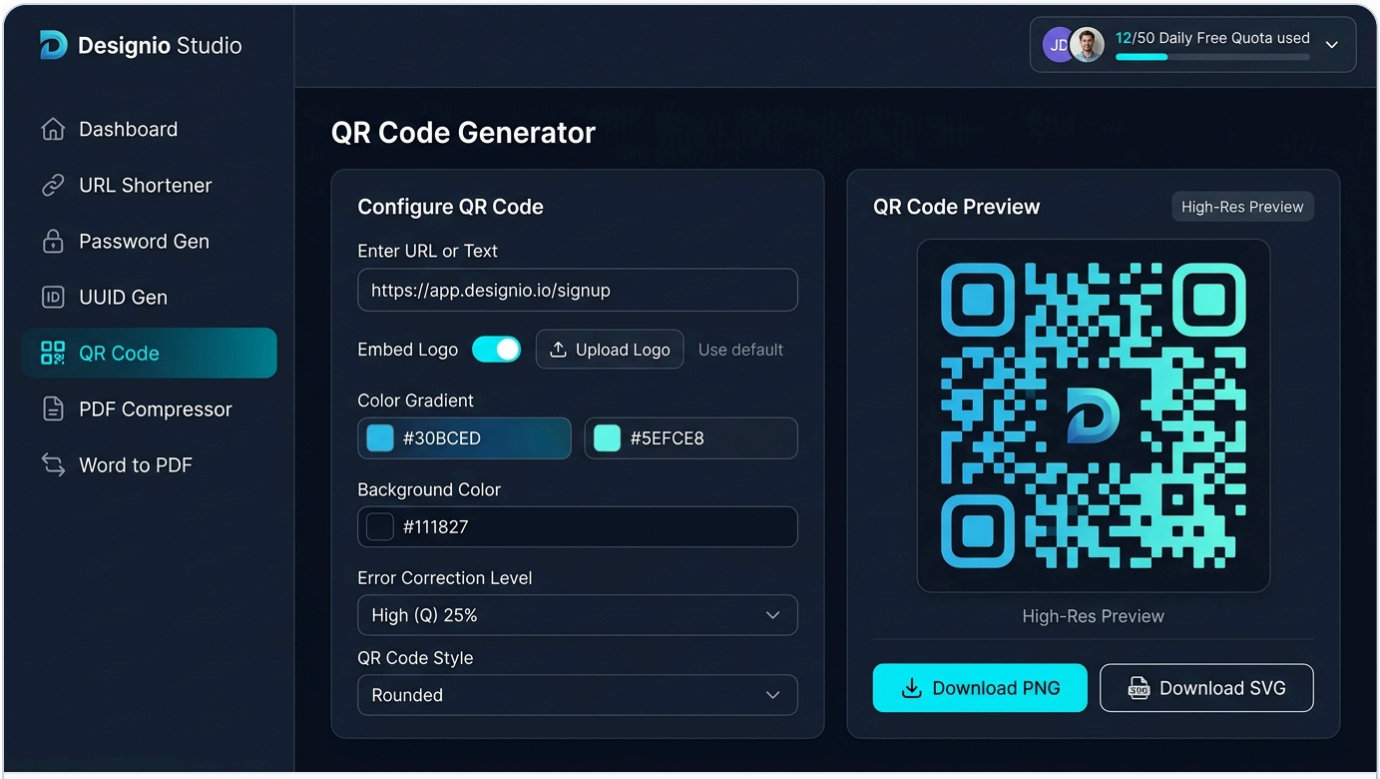
Screen 3: Dual-Track Pricing Matrix with Interactive Cost Slider ROUTE: /PRICING

Figma Component Structure:

- **Segmented Control:** Seamless animated toggle switching between `Normal User Plans` and `Developer API Plans`.
- **Developer Tier Cards:**
  - **Starter (Free):** 1,000 requests/month, 1 API key, 10 req/min rate limit, community support.
  - **Growth ($29/mo):** 100,000 requests/month, 5 API keys, 60 req/sec burst limit, Webhook alerts, priority support.
  - **Enterprise ($99/mo):** 1,000,000 requests/month, 300 req/sec rate limit, custom short URL domains, 99.99% SLA.
- **Interactive API Cost Estimator:** Draggable volume slider that recalculates expected monthly cost in real time.

Screen 4: Normal User Tool Studio (Browser Tools)

A distraction-free, focused workspace for end-users to generate QR codes, convert documents, and compress PDFs.



Screen 4: Normal User Tool Workspace (e.g., QR Code Studio) ROUTE: /TOOLS/QR

Tool-by-Tool Figma Screen Specifications:

1. QR Code Studio `/tools/qr`

Input field for URL or text, toggle to upload/embed custom brand logo, hex color gradient picker, error correction level dropdown (L, M, Q, H), and instant PNG/SVG download.

2. URL Shortener `/tools/url`

Long URL input, optional custom vanity alias, expiration date picker, generated short link with 1-click copy, and preview of visit analytics.

3. PDF Compressor `/tools/pdf-compressor`

Drag-and-drop PDF dropzone, compression quality presets (Low, Medium, High, Extreme), before/after file size reduction meter, and instant download.

4. Word to PDF `/tools/word-to-pdf`

DOCX file upload area, conversion status progress bar, PDF thumbnail preview, and download button.

5. Image to PDF `/tools/img-to-pdf`

6. Password Generator `/tools/password`

Multi-image upload with thumbnail re-ordering via drag-and-drop, margin controls (None, Small, Big), orientation (Portrait/Landscape), and PDF merge button.

Character length slider (8 to 64), toggles for uppercase, numbers, and symbols, entropy strength bar (Weak to Very Strong), and 1-click clipboard copy.

7. UUID Generator `/tools/uuid`

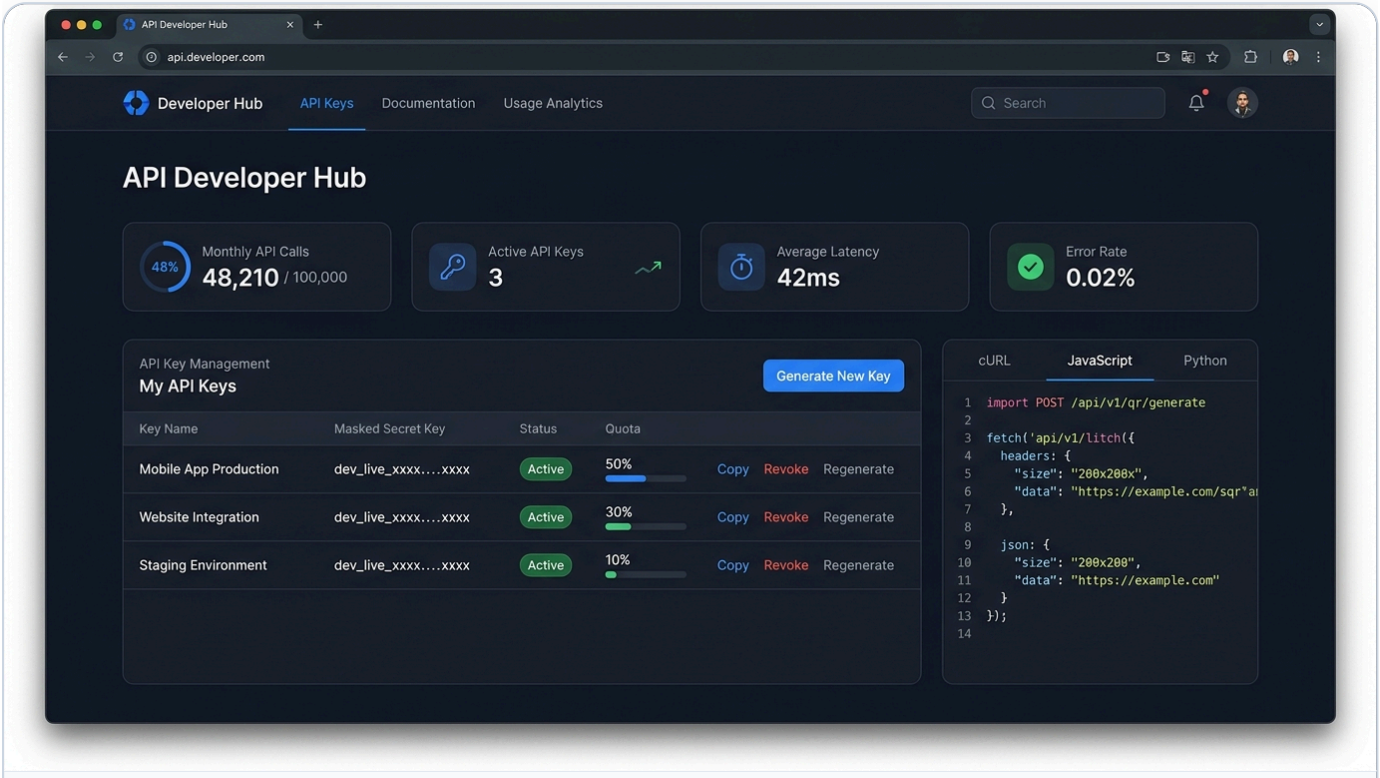
Version selector (UUID v4 Random, UUID v7 Time-Ordered), bulk count generator (1 to 100), uppercase toggle, and copy-all button.

8. Image Placeholder `/tools/placeholder`

Width and Height dimensions, custom label text, background and text color pickers, and live canvas preview with direct copy URL or PNG export.

Screen 5: Developer Portal & API Key Management Hub

Empowers software engineers to integrate DevToolBox capabilities directly into their software without writing tools from scratch.



Screen 5: Developer Hub with API Key Management and Live Code Sandbox

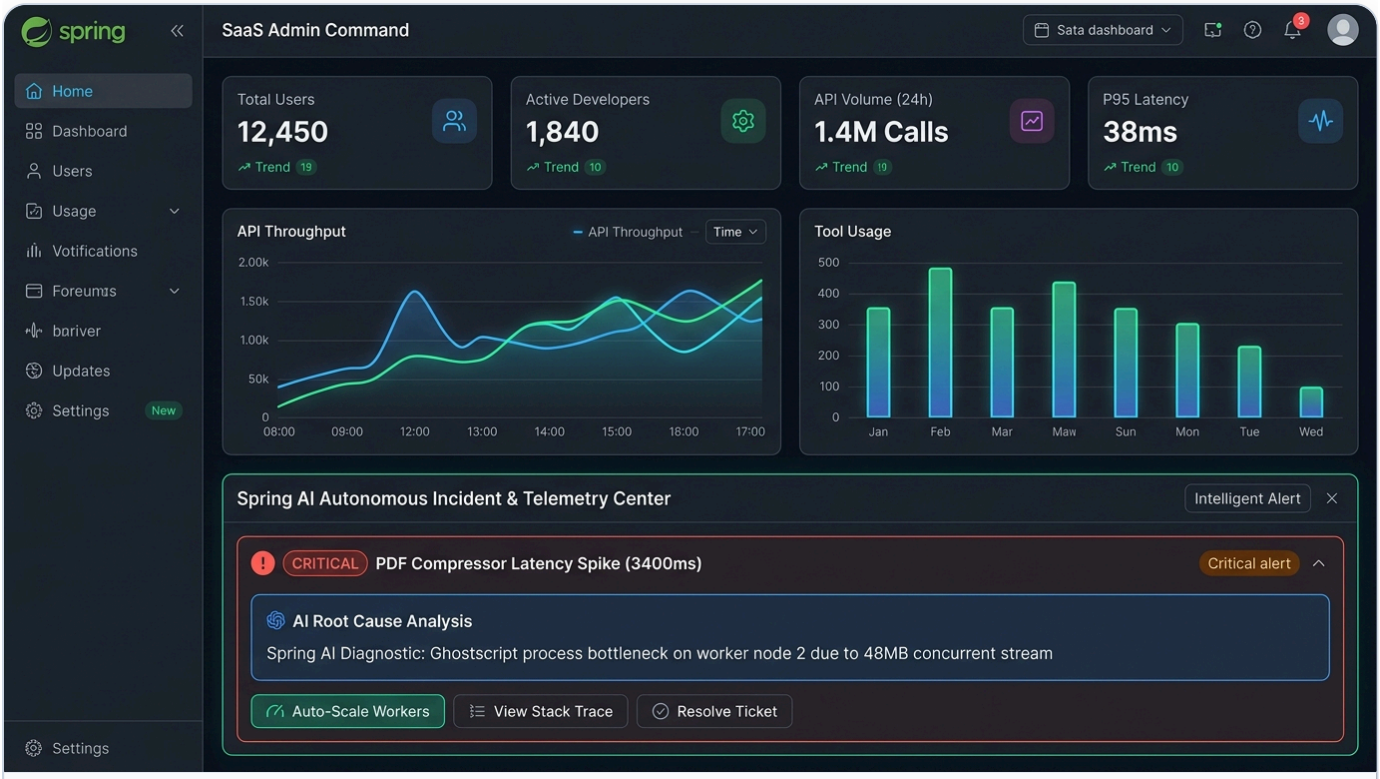
ROUTE: /DEVELOPER

Figma Component Structure:

- **Telemetry Stat Cards:** Monthly API Calls (circular progress gauge), Active Keys count, Average Latency (42ms), Error Rate (0.02%).
- **API Key Table:** Key Name (e.g. `Mobile App Production`), Masked Secret Key (`dev_live_xxxx...xxxx`), Status badge (`Active`), Monthly quota bar, and action links: `Copy`, `Revoke`, `Regenerate`.
- **Interactive Code Playground:** Multi-language tab switcher (`cURL`, `JavaScript`, `Python`, `Java`) generating ready-to-run code snippets pre-filled with the developer's API Key.

Screen 6: Admin Command Center & Spring AI Observability

Complete administrative control over users, developers, API traffic, and autonomous AI-driven incident management.



Screen 6: Admin Telemetry Dashboard & Spring AI Autonomous Incident Center ROUTE: /ADMIN

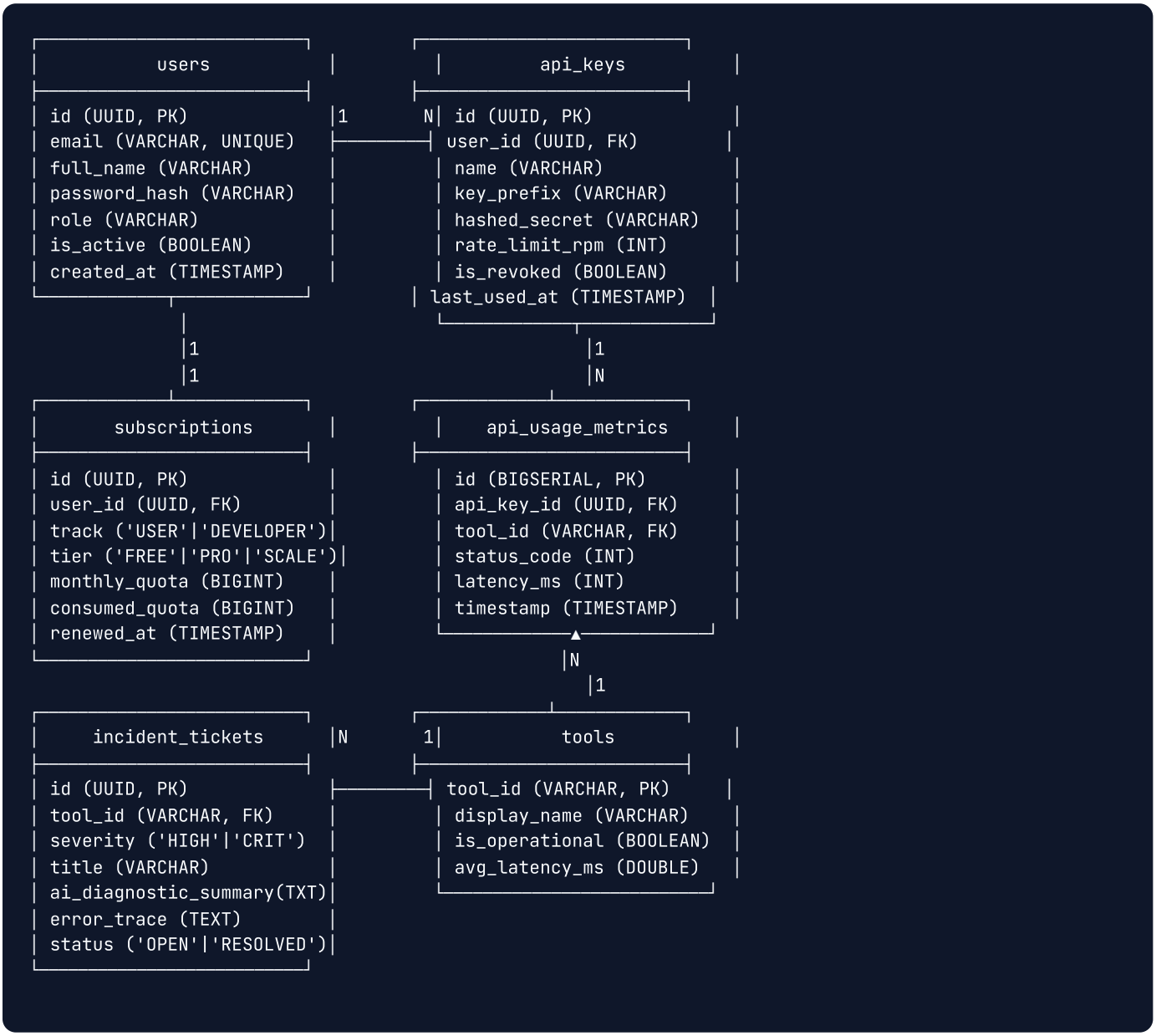
Figma Component Structure:

- **Global KPI Strip:** Total Registered Users, Active Developers, API Volume (24h), and P95 System Latency.
- **Analytics Charts (Recharts):** Throughput line chart (requests/min) and Tool Usage breakdown bar chart.
- **Spring AI Autonomous Incident Center:**
  - Auto-generated alert cards triggered whenever a tool experiences an unhandled exception or latency > 1500ms.
  - **AI Root Cause Analysis:** Plain-English summary explaining the underlying issue (e.g., *"Ghostscript process bottleneck on worker node 2 due to 48MB concurrent stream"*).
  - **Action Buttons:** `Auto-Scale Workers`, `View Stack Trace`, and `Resolve Ticket`.

3. Database Entities & Entity-Relationship Architecture

The target database is **PostgreSQL** managed through versioned **Flyway** scripts. High-throughput caching and rate limiting are offloaded to **Redis**.

3.1 PostgreSQL Relational Schema



3.2 Redis High-Performance Caching & Quota Strategy

KEY PATTERN	DATA TYPE	TTL	PURPOSE & BEHAVIOR
`ratelimit:{api_key}:{minute_bucket}`	String (Counter)	60 seconds	Atomic Redis `INCR` command. Rejects request with `HTTP 429` if count > tier limit.
`quota:developer:{api_key_id}`	Hash	30 days	Tracks consumed monthly API call credits. Decremented on every valid execution.
`url:redirect:{short_code}`	String	7 days	Ultra-fast redirect lookup (< 2ms) without querying PostgreSQL on every link visit.
`otp:auth:{email}`	String	5 minutes	Secure, time-expiring OTP for password reset (eliminates in-memory Java HashMap).

4. Spring Boot 3.4 & Spring AI Backend Architecture

4.1 Spring AI Autonomous Incident Diagnostics

Unlike a simple chatbot, **Spring AI** is wired directly into the Spring Boot application's exception and telemetry pipelines. It operates as an autonomous diagnostic agent.

> **Autonomous Triage Workflow**
>
> When an unhandled exception occurs in a tool controller (e.g. `PdfCompressor`) or latency exceeds 1500ms, an `ApplicationEvent` triggers the `SpringAiIncidentService`. The service sends the stack trace, memory metrics, and input metadata to the Gemini model, which analyzes the failure and persists an **Incident Ticket** in PostgreSQL.

```
@Service
@RequiredArgsConstructor
@Slf4j
public class SpringAiIncidentService {

    private final ChatClient chatClient;
    private final IncidentTicketRepository ticketRepo;

    @Async
    @EventListener
    public void onToolFailure(ToolExecutionFailureEvent event) {
        String prompt = """
            You are DevToolBox Autonomous Site Reliability Engineer.
            Analyze this production failure on tool: %s.
            Error: %s
            Stack Trace: %s
            Provide:
            1. Root Cause Summary (1-2 sentences)
            2. Severity (LOW, MEDIUM, HIGH, CRITICAL)
            3. Recommended Action for the Admin
            """.formatted(event.getToolId(), event.getErrorMessage(), event.getStackTrace());

        String aiAnalysis = chatClient.prompt(prompt).call().content();

        IncidentTicket ticket = IncidentTicket.builder()
            .toolId(event.getToolId())
            .title("Automated Alert: Failure on " + event.getToolId())
            .severity(determineSeverity(event))
            .aiDiagnosticSummary(aiAnalysis)
            .errorTrace(event.getStackTrace())
            .status(TicketStatus.OPEN)
            .createdAt(Instant.now())
            .build();

        ticketRepo.save(ticket);
        log.warn("Spring AI created incident ticket: {}", ticket.getId());
    }
```

```
}
```

4.2 API Key Filter Chain (`X-API-KEY`)

The backend supports two distinct authentication pathways in `SecurityConfig`:

- **Browser Sessions:** Authenticated via `Authorization: Bearer <JWT>` header.
- **Developer API Calls:** Authenticated via `X-API-KEY: dev_live_...` header. The filter hashes the incoming key using SHA-256, looks up the active API key in PostgreSQL/Redis, increments the rate-limit counter, and injects `ROLE_DEVELOPER` into the Spring Security Context.

5. The "React Pro" Mentorship Framework & Project Structure

To ensure you master React completely, our collaboration follows a strict pair-programming dynamic: **You write the code, and I review, explain, and guide you through industry best practices.**

5.1 React Pro Learning Curriculum

MODULE	CORE CONCEPTS YOU WILL MASTER	PROJECT DELIVERABLE
Module 1: Architecture & State	Container vs Presentational components, avoiding prop drilling, memoization (`useMemo` , `useCallback`).	`Landing.jsx` , `Navbar.jsx` , `Pricing.jsx`
Module 2: Custom Hooks	Extracting business logic, async loading/error states, race condition prevention with cleanup functions.	`useApiKey()` , `useRateLimit()` , `useToolRunner()`
Module 3: Auth & Security	Axios interceptors, silent token refresh, role-guarded routes (`<RequireRole role="DEVELOPER" />`).	`AuthContext.jsx` , `Login.jsx` , `SignUp.jsx`
Module 4: UI & Micro-interactions	Tailwind v4 tokens, responsive layouts, animated modals, drag-and-drop file dropzones with Framer Motion.	Tool Studios (QR, PDF, URL, Password)
Module 5: Developer Tooling	Copy-to-clipboard, syntax-highlighted code generator, dynamic request sandbox.	`DeveloperHub.jsx` , `ApiDocs.jsx`
Module 6: Observability UI	Recharts data visualization, real-time ticket polling, administrative table filtering.	`AdminDashboard.jsx` , `Incidents.jsx`

5.2 Step-by-Step Implementation Roadmap

Phase 1: Project Setup & Auth Core

1. Scaffold Spring Boot 3.4 Maven project with dependencies (Security, JPA, Redis, Flyway, Spring AI).
2. Configure Flyway migration scripts (`V1__schema.sql`).
3. Implement JWT + API Key filter chain in Spring Security.
4. **React Mentoring:** Build `AuthContext` and `Landing.jsx` with role selector.

Phase 2: Pricing & Developer Hub

1. Backend: API Key generation service, SHA-256 hashing, Redis Token Bucket rate limiter.
2. Backend: Developer telemetry logging filter.
3. **React Mentoring:** Build `Pricing.jsx` dual-track matrix and `DeveloperHub.jsx` with key generation table and code snippets.

Phase 3: Tool Studios Overhaul

1. Backend: Dockerize PDF compression and DOCX conversion (Ghostscript + LibreOffice).
2. Backend: Clean REST endpoints for QR, URL Shortener, and Password/UUID tools.
3. **React Mentoring:** Build interactive tool studios with drag-and-drop file uploaders and preview states.

Phase 4: Admin & Spring AI Observability

1. Backend: Spring AI incident diagnostic service and telemetry aggregate APIs.
2. **React Mentoring:** Build `AdminDashboard.jsx` with Recharts telemetry and Spring AI Incident ticket workflow.

Ready for Execution

This complete document and all associated Figma screen blueprints establish our master specification. You can now begin coding the first component!